Advanced SQL Query Constructs

Michel Ferreira

Bases de Dados

Licenciatura em Engenharia Informática e Computação, FEUP+FCUP

What is a CTE?

CTE = Common Table Expression

A temporary, named result set defined at the start of a query using WITH.

Used to:

- Improve readability of complex queries
- Break logic into steps
- Avoid repeating subqueries

Example structure:

```
WITH cte_name AS (
    SELECT ...
)
SELECT ...
FROM cte_name;
```

Basic Syntax

```sql
WITH cte_name AS (
    SELECT col1, col2
    FROM table
    WHERE condition
)
SELECT *
FROM cte_name
WHERE ...
```

Key points:

- Exists only during execution of the statement
- Behaves like a temporary result table
- The final query can reference one or more CTEs

Multiple CTEs

```sql
WITH
  a AS (SELECT ...),
  b AS (SELECT ...),
  c AS (
      SELECT ...
      FROM a JOIN b
  )
SELECT *
FROM c;
```

Notes:

- CTEs can reference earlier CTEs
- They help organize multi-step logic
- Replace deeply nested subqueries

An Example with Autárquicas 2021 Database

Question:

Using the Autárquicas 2021 database, write an SQL query (using a CTE) that lists the winning list in each municipality of the Autonomous Region of Madeira.

Consider party lists, coalition lists, and independent lists.

Your query should return, for each municipality in Madeira, the municipality name, the short name of the winning list, its type (party, coalition, or independent), and the number of votes it obtained. Only results for the Autonomous Region of Madeira should be included.

Example Solution

```sql
WITH all_lists AS (
    -- Parties
    SELECT
        pv.municipality_code,
        p.shortname AS shortname,
        pv.votes    AS votes,
        'party'     AS list_type
    FROM party_votes pv
    JOIN party p ON p.id = pv.party_id

    UNION ALL

    -- Coalitions
    SELECT
        c.municipality_code,
        c.shortname,
        c.votes,
        'coalition'
    FROM coalition c

    UNION ALL

    -- Independents
    SELECT
        i.municipality_code,
        i.shortname,
        i.votes,
        'independent'
    FROM independent i
)
```

```sql
SELECT
    m.name AS municipality,
    a.shortname AS winner,
    a.list_type,
    a.votes
FROM municipality m
JOIN district d
  ON d.id = m.district
JOIN all_lists a
  ON a.municipality_code = m.code
WHERE d.name = 'Madeira'
  AND a.votes = (
      SELECT MAX(votes)
      FROM all_lists
      WHERE municipality_code = m.code
  )
ORDER BY m.name, a.shortname;
```

municipality	winner	list_type	votes
CALHETA	PPD/PSD	party	5047
CÂMARA DE LOBOS	PPD/PSD	party	9866
FUNCHAL	PPD/PSD.CDS-PP	coalition	26835
MACHICO	PS	party	5951
PONTA DO SOL	PS	party	2978
PORTO MONIZ	PS	party	1147
PORTO SANTO	PPD/PSD.CDS-PP	coalition	1843
RIBEIRA BRAVA	RB1	independent	4698
SANTA CRUZ	JPP	party	11932
SANTANA	CDS-PP	party	2720
SÃO VICENTE	PPD/PSD.CDS-PP	coalition	2343

Example with Two CTEs

Query: in Madeira, find each municipality's winner (CTE #1), then summarize how many municipalities each list won (CTE #2).

```sql
WITH all_lists AS (
    -- unify parties, coalitions, and independents into one stream
    SELECT pv.municipality_code, p.shortname, pv.votes, 'party' AS list_type
    FROM party_votes pv
    JOIN party p ON p.id = pv.party_id

    UNION ALL
    SELECT c.municipality_code, c.shortname, c.votes, 'coalition'
    FROM coalition c

    UNION ALL
    SELECT i.municipality_code, i.shortname, i.votes, 'independent'
    FROM independent i
),
```

```sql
winners_madeira AS (
    -- pick the top-voted list(s) in each Madeira municipality
    SELECT
        m.code AS municipality_code,
        m.name AS municipality,
        a.shortname,
        a.list_type,
        a.votes
    FROM municipality m
    JOIN district d ON d.id = m.district
    JOIN all_lists a ON a.municipality_code = m.code
    WHERE d.name = 'Madeira'
      AND a.votes = (
          SELECT MAX(votes)
          FROM all_lists
          WHERE municipality_code = m.code
      )
)
```

Example with Two CTEs (cont.)

```sql
SELECT
    shortname AS winner,
    list_type,
    COUNT(*) AS municipalities_won
FROM winners_madeira
GROUP BY shortname, list_type
ORDER BY municipalities_won DESC, shortname;
```

winner	list_type	municipalities_won
PPD/PSD.CDS-PP	coalition	3
PS	party	3
PPD/PSD	party	2
CDS-PP	party	1
JPP	party	1
RB1	independent	1

What each CTE does:

- all_lists: creates one unified table of all vote sources (party/coalition/independent).
- winners_madeira: filters to Madeira and keeps only the list(s) with max votes per municipality.
- Final query: counts wins per list in Madeira.

Recursive CTEs (WITH RECURSIVE)

Purpose:

Recursive CTEs allow a query to repeatedly apply a rule to generate successive rows, typically used for hierarchical data, graph traversal, or generating numeric sequences.

```sql
WITH RECURSIVE cte_name AS (
    -- 1. Anchor member: produces the initial row(s)
    SELECT ...

    UNION ALL

    -- 2. Recursive member: refers to the CTE itself
    SELECT ...
    FROM cte_name
    WHERE ...
)
SELECT * FROM cte_name;
```

- The anchor member starts the recursion.
- The recursive member repeatedly builds on previous results.
- Recursion ends when the recursive member produces no new rows.

Example: generate vote thresholds (50k - 500k)

Analyse how many parties exceed increasing vote thresholds.

- Builds the sequence 50000, 100000, …, 500000.
- For each threshold t, counts how many party lists nationwide received more than t votes.

```sql
WITH RECURSIVE thresholds AS (
    -- Anchor: start at 50,000 votes
    SELECT 50000 AS t

    UNION ALL

    -- Recursive step: add 50,000 until reaching 500,000
    SELECT t + 50000
    FROM thresholds
    WHERE t < 500000
),
party_totals AS (
    -- Total votes per party across ALL municipalities (national total)
    SELECT
        pv.party_id,
        SUM(pv.votes) AS total_votes
    FROM party_votes pv
    GROUP BY pv.party_id
)
```

```sql
SELECT
    th.t AS vote_threshold,
    COUNT(*) AS parties_above_threshold
FROM thresholds th
JOIN party_totals pt
  ON pt.total_votes > th.t
GROUP BY th.t
ORDER BY th.t;
```

vote_threshold	parties_above_threshold
50000	6
100000	5
150000	3
200000	3
250000	3
300000	3
350000	2
400000	1
450000	1
500000	1

Using a Recursive Query to Evaluate Mandates Distribution using the D’Hondt Method

- In many elections, such as in Autárquicas, the mandates for the Municipal Council are allocated using the D'Hondt method, which assigns seats one by one based on descending vote quotients.
- Let’s build a recursive D’Hondt (Hondt) query for computing the mandates distribution for the Municipality of ‘PORTO’.
- Let’s start by getting all lists and votes in ‘PORTO’.
- We first unify parties, coalitions, and independents into one list of competitors for ‘PORTO’.

D’Hondt Method Recursive Query

- Step 1) Get all lists and votes in ‘PORTO’

```sql
WITH lists_porto AS (
    -- Parties in Porto
    SELECT
        m.code AS municipality_code,
        p.shortname AS shortname,
        pv.votes AS votes,
        'party' AS list_type
    FROM municipality m
    JOIN party_votes pv ON pv.municipality_code = m.code
    JOIN party p ON p.id = pv.party_id
    WHERE m.name = 'PORTO'

    UNION ALL

    -- Coalitions in Porto
    SELECT
        m.code,
        c.shortname,
        c.votes,
        'coalition'
    FROM municipality m
    JOIN coalition c ON c.municipality_code = m.code
    WHERE m.name = 'PORTO'

    UNION ALL

    -- Independents in Porto
    SELECT
        m.code,
        i.shortname,
        i.votes,
        'independent'
    FROM municipality m
    JOIN independent i ON i.municipality_code = m.code
    WHERE m.name = 'PORTO'
)
SELECT * FROM lists_porto
ORDER BY votes DESC, shortname;
```

municipality_code	shortname	votes	list_type
131200	RM	41213	independent
131200	PS	18192	party
131200	PPD/PSD	17481	party
131200	PCP-PEV	7610	party
131200	BE	6321	party
131200	CH	2983	party
131200	PAN	2821	party
131200	L	461	party
131200	VP	424	party
131200	PPM	211	party
131200	E	79	party

D’Hondt Method Recursive Query (cont. I)

- Step 2) Get how many mandates ’PORTO’ elects

```sql
WITH porto_mandates AS (
    SELECT mandates
    FROM municipality
    WHERE name = 'PORTO'
)
SELECT * FROM porto_mandates;
```

```
mandates
--------
13
```

D’Hondt Method Recursive Query (cont. II)

- Step 3 — Using a Recursive CTE to Generate the D’Hondt Divisors
- In D’Hondt, each list competes with the sequence of divisors:

$$1,\ 2,\ 3,\ \ldots,\ M$$

  where M is the number of mandates in the municipality.
- We need these numbers (k) because each quotient is:

$$\text{quotient} = \frac{\text{votes}}{k}$$

- So our next step is to generate the divisor sequence 1, 2, 3, …, mandates.
- We do this with a recursive CTE.

D’Hondt Method Recursive Query (cont. III)

- Recursive CTE to generate the divisors

```sql
divisors AS (
    -- Anchor: start at 1
    SELECT 1 AS k
    UNION ALL
    -- Recursive step: 2,3,...,n (n = mandates)
    SELECT k + 1
    FROM divisors
    WHERE k < (SELECT n FROM porto_mandates)
)
```

- The anchor row produces the first divisor: 1.
- The recursive part keeps adding 1 until reaching the mandate count.
- For ‘PORTO’ (13 mandates), this emits:
- 1, 2, 3, ..., 13.
- This is the only part of the query that needs recursion.

D’Hondt Method Recursive Query (cont. IV)

- Step 4 — Generate all D’Hondt quotients
  - Once we have lists_porto and divisors, we take the Cartesian combination:

```
SELECT
    lp.shortname,
    lp.list_type,
    d.k AS divisor,
    (lp.votes * 1.0) / d.k AS quotient
FROM lists_porto lp
JOIN divisors d
```

  - We generate all possible quotient candidates for seats.
- Step 5 — Select the top N quotients (N = number of mandates)

```
SELECT shortname, list_type, divisor, quotient
FROM quotients
ORDER BY quotient DESC, shortname ASC, divisor ASC
LIMIT (SELECT n FROM porto_mandates)
```

D’Hondt Method Recursive Query (cont. V)

- Step 6 — Count seats per list
  - Finally:

```sql
SELECT
    shortname,
    list_type,
    COUNT(*) AS mandates_elected
FROM top_quotients
GROUP BY shortname, list_type
ORDER BY mandates_elected DESC, shortname ASC;
```

  - Each occurrence of a list in the top quotients = 1 elected mandate.
  - Group and count: gives the final distribution.
- This produces the correct D’Hondt allocation for the Municipality of Porto, using a recursive CTE only where it is logically needed (to generate the divisor sequence).

D’Hondt Method Recursive Query (cont. VI)

```sql
WITH RECURSIVE
lists_porto AS (
    -- Parties
    SELECT p.shortname, pv.votes, 'party' AS list_type
    FROM municipality m
    JOIN party_votes pv ON pv.municipality_code = m.code
    JOIN party p ON p.id = pv.party_id
    WHERE m.name = 'PORTO'

    UNION ALL
    -- Coalitions
    SELECT c.shortname, c.votes, 'coalition'
    FROM municipality m
    JOIN coalition c ON c.municipality_code = m.code
    WHERE m.name = 'PORTO'

    UNION ALL
    -- Independents
    SELECT i.shortname, i.votes, 'independent'
    FROM municipality m
    JOIN independent i ON i.municipality_code = m.code
    WHERE m.name = 'PORTO'
),
```

```sql
porto_mandates AS (
    SELECT mandates AS n
    FROM municipality
    WHERE name = 'PORTO'
),
divisors AS (
    -- Anchor: start at 1
    SELECT 1 AS k
    UNION ALL
    -- Recursive step: 2,3,...,n
    SELECT k + 1
    FROM divisors
    WHERE k < (SELECT n FROM porto_mandates)
),
quotients AS (
    -- All D'Hondt quotients for Porto
    SELECT
        lp.shortname,
        lp.list_type,
        d.k AS divisor,
        (lp.votes * 1.0) / d.k AS quotient
    FROM lists_porto lp
    JOIN divisors d
),
```

```sql
top_quotients AS (
    -- Keep only the top N quotients (= mandates)
    SELECT shortname, list_type, divisor, quotient
    FROM quotients
    ORDER BY quotient DESC, shortname ASC, divisor ASC
    LIMIT (SELECT n FROM porto_mandates)
)
SELECT
    shortname,
    list_type,
    COUNT(*) AS mandates_elected
FROM top_quotients
GROUP BY shortname, list_type
ORDER BY mandates_elected DESC, shortname ASC;
```

shortname	list_type	mandates_elected
RM	independent	6
PS	party	3
PPD/PSD	party	2
BE	party	1
PCP-PEV	party	1

Window Functions

Core idea:

- A window function computes a value for each row, using a related set of rows (the window).
- Unlike GROUP BY, it does not collapse rows (it keeps detail).

When to use:

- Rankings (“top N per group”), running totals, moving averages
- Compare to previous/next row (lag/lead)
- Relative position / distribution (percent_rank, cume_dist)
- Rule-based de-duplication (e.g., “latest record per customer”)

Quick mantra

- GROUP BY → reduces rows
- OVER (...) → enriches rows

Basic syntax: OVER (PARTITION BY … ORDER BY …)

General form:

- function(...) OVER (PARTITION BY ... ORDER BY ... [frame])

Pieces:

- PARTITION BY: defines the group (e.g., per customer, district, department)
- ORDER BY: defines the order within the group (crucial for running totals, lag/lead, rankings)
- Frame (later): defines “which rows count” at each row

Example

```sql
WITH orders AS (
  SELECT *
  FROM (VALUES
    (1, 'Ana',   TIMESTAMP '2026-01-03 10:00', 20),
    (2, 'Ana',   TIMESTAMP '2026-01-10 09:30', 35),
    (3, 'Ana',   TIMESTAMP '2026-01-10 18:00', 15),
    (4, 'Bruno', TIMESTAMP '2026-01-02 11:00', 40),
    (5, 'Bruno', TIMESTAMP '2026-01-05 15:00', 10),
    (6, 'Bruno', TIMESTAMP '2026-01-12 12:00', 25),
    (7, 'Carla', TIMESTAMP '2026-01-04 08:00', 50),
    (8, 'Carla', TIMESTAMP '2026-01-15 19:00', 30)
  ) AS t(order_id, customer, order_ts, amount)
)
SELECT
  order_id, customer, order_ts, amount,
  SUM(amount) OVER (PARTITION BY customer) AS customer_total
FROM orders
ORDER BY customer, order_ts, order_id;
```

Window vs GROUP BY: keep the rows

Goal: show totals and keep each order row.

GROUP BY collapses

```sql
WITH orders AS (
  SELECT *
  FROM (VALUES
    (1, 'Ana', 20),
    (2, 'Ana', 35),
    (3, 'Bruno', 40),
    (4, 'Bruno', 10)
  ) AS t(order_id, customer, amount)
)
SELECT customer, SUM(amount) AS customer_total
FROM orders
GROUP BY customer
ORDER BY customer;
```

Window keeps detail

```sql
WITH orders AS (
  SELECT *
  FROM (VALUES
    (1, 'Ana', 20),
    (2, 'Ana', 35),
    (3, 'Bruno', 40),
    (4, 'Bruno', 10)
  ) AS t(order_id, customer, amount)
)
SELECT
  order_id, customer, amount,
  SUM(amount) OVER (PARTITION BY customer) AS customer_total
FROM orders
ORDER BY customer, order_id;
```

Ranking functions: row_number, rank, dense_rank, ntile

What they do:

- row_number() → unique sequence per partition
- rank() → ties share rank, leaves gaps (1,1,3,…)
- dense_rank() → ties share rank, no gaps (1,1,2,…)
- ntile(n) → bucket rows into n groups

```sql
WITH sales AS (
  SELECT *
  FROM (VALUES
    ('North', 'A', 100),
    ('North', 'B', 100),
    ('North', 'C',  80),
    ('South', 'D', 120),
    ('South', 'E',  90),
    ('South', 'F',  90)
  ) AS t(region, rep, revenue)
)
SELECT
  region, rep, revenue,
  ROW_NUMBER()  OVER (PARTITION BY region ORDER BY revenue DESC, rep) AS rn,
  RANK()        OVER (PARTITION BY region ORDER BY revenue DESC)      AS rnk,
  DENSE_RANK()  OVER (PARTITION BY region ORDER BY revenue DESC)      AS drnk,
  NTILE(2)      OVER (PARTITION BY region ORDER BY revenue DESC)      AS half_bucket
FROM sales
ORDER BY region, revenue DESC, rep;
```

```
 region | rep | revenue | rn | rnk | drnk | half_bucket
--------+-----+---------+----+-----+------+-------------
 North  | A   |     100 |  1 |   1 |    1 |           1
 North  | B   |     100 |  2 |   1 |    1 |           1
 North  | C   |      80 |  3 |   3 |    2 |           2
 South  | D   |     120 |  1 |   1 |    1 |           1
 South  | E   |      90 |  2 |   2 |    2 |           1
 South  | F   |      90 |  3 |   2 |    2 |           2
(6 rows)
```

Running totals and running counts (ordered windows)

Key:

- Add ORDER BY inside OVER to make the window “progress along time”.

```sql
WITH orders AS (
  SELECT *
  FROM (VALUES
    (1, 'Ana',   DATE '2026-01-03', 20),
    (2, 'Ana',   DATE '2026-01-10', 35),
    (3, 'Ana',   DATE '2026-01-10', 15),
    (4, 'Bruno', DATE '2026-01-02', 40),
    (5, 'Bruno', DATE '2026-01-05', 10),
    (6, 'Bruno', DATE '2026-01-12', 25)
  ) AS t(order_id, customer, order_date, amount)
)
SELECT
  order_id, customer, order_date, amount,
  SUM(amount)  OVER (PARTITION BY customer ORDER BY order_date, order_id) AS running_total,
  COUNT(*)     OVER (PARTITION BY customer ORDER BY order_date, order_id) AS running_count
FROM orders
ORDER BY customer, order_date, order_id;
```

order_id	customer	order_date	amount	running_total	running_count
1	Ana	2026-01-03	20	20	1
2	Ana	2026-01-10	35	55	2
3	Ana	2026-01-10	15	70	3
4	Bruno	2026-01-02	40	40	1
5	Bruno	2026-01-05	10	50	2
6	Bruno	2026-01-12	25	75	3

(6 rows)

Window frames

Frame controls which rows are aggregated at each row:

- ROWS → physical row offsets (exactly N rows)
- RANGE → value-based range around the ordering value (can include ties)
- GROUPS → groups of peers (ties) as units (advanced)

3-row moving average (physical rows):

```sql
WITH prices AS (
  SELECT *
  FROM (VALUES
    ('A', DATE '2026-01-01', 100),
    ('A', DATE '2026-01-02', 105),
    ('A', DATE '2026-01-03', 102),
    ('A', DATE '2026-01-04', 110),
    ('A', DATE '2026-01-05', 108)
  ) AS t(symbol, d, price)
)
SELECT
  symbol, d, price,
  AVG(price) OVER (
    PARTITION BY symbol ORDER BY d
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  ) AS ma_3rows
FROM prices
ORDER BY symbol, d;
```

```
 symbol |      d       | price |        ma_3rows
--------+--------------+-------+----------------------
 A      | 2026-01-01 |   100 | 100.0000000000000000
 A      | 2026-01-02 |   105 | 102.5000000000000000
 A      | 2026-01-03 |   102 | 102.3333333333333333
 A      | 2026-01-04 |   110 | 105.6666666666666667
 A      | 2026-01-05 |   108 | 106.6666666666666667
(5 rows)
```

Percentiles and distribution: percent_rank, cume_dist

Use cases:

- “Top 10% customers”, outlier detection, relative performance metrics.

```sql
WITH customers AS (
  SELECT *
  FROM (VALUES
    ('Ana',   70),
    ('Bruno', 40),
    ('Carla', 90),
    ('Duarte',90),
    ('Eva',   55)
  ) AS t(customer, score)
)
SELECT
  customer, score,
  PERCENT_RANK() OVER (ORDER BY score) AS percent_rank,
  CUME_DIST()    OVER (ORDER BY score) AS cume_dist
FROM customers
ORDER BY score, customer;
```

```
 customer | score | percent_rank | cume_dist
----------+-------+--------------+-----------
 Bruno    |    40 |            0 |       0.2
 Eva      |    55 |         0.25 |       0.4
 Ana      |    70 |          0.5 |       0.6
 Carla    |    90 |         0.75 |         1
 Duarte   |    90 |         0.75 |         1
(5 rows)
```

Interpretation:

- percent_rank → 0 for the lowest row; approaches 1 for the highest (depends on N)
- cume_dist → fraction of rows with value ≤ current row (ties jump)

FILTER: conditional aggregation

What it is:

- Applies a condition to one aggregate only.
- Cleaner than embedding CASE WHEN inside SUM/COUNT/AVG.

Syntax:

```
agg(expr) FILTER (WHERE condition)
```

Example: multiple conditional counts in one GROUP BY:

```sql
WITH trips AS (
  SELECT *
  FROM (VALUES
    ('Porto',  'cash',   12.5),
    ('Porto',  'card',   18.0),
    ('Porto',  'card',    9.2),
    ('Lisbon', 'cash',   22.0),
    ('Lisbon', 'credit', 15.0),
    ('Lisbon', 'card',   11.5),
    ('Lisbon', 'credit', 30.0)
  ) AS t(city, pay_method, amount)
)
SELECT
  city,
  COUNT(*) AS total_trips,
  COUNT(*) FILTER (WHERE pay_method = 'cash')   AS cash_trips,
  COUNT(*) FILTER (WHERE pay_method = 'card')   AS card_trips,
  COUNT(*) FILTER (WHERE pay_method = 'credit') AS credit_trips
FROM trips
GROUP BY city
ORDER BY city;
```

```
  city   | total_trips | cash_trips | card_trips | credit_trips
--------+-------------+------------+------------+--------------
 Lisbon |           4 |          1 |          1 |            2
 Porto  |           3 |          1 |          2 |            0
(2 rows)
```

FILTER + window functions (conditional totals per row)

Goal:

- Keep row detail, but add group totals and conditional group totals.

```sql
WITH trips AS (
  SELECT *
  FROM (VALUES
    (1, 'Porto',  'cash',   12.5),
    (2, 'Porto',  'card',   18.0),
    (3, 'Porto',  'card',    9.2),
    (4, 'Lisbon', 'cash',   22.0),
    (5, 'Lisbon', 'credit', 15.0),
    (6, 'Lisbon', 'card',   11.5)
  ) AS t(trip_id, city, pay_method, amount)
)
SELECT
  trip_id, city, pay_method, amount,
  SUM(amount) OVER (PARTITION BY city) AS city_total_revenue,
  SUM(amount) FILTER (WHERE pay_method = 'card')
    OVER (PARTITION BY city)           AS city_card_revenue,
  COUNT(*) FILTER (WHERE pay_method IN ('card','credit'))
    OVER (PARTITION BY city)           AS city_electronic_trips
FROM trips
ORDER BY city, trip_id;
```

trip_id	city	pay_method	amount	city_total_revenue	city_card_revenue	city_electronic_trips
4	Lisbon	cash	22.0	48.5	11.5	2
5	Lisbon	credit	15.0	48.5	11.5	2
6	Lisbon	card	11.5	48.5	11.5	2
1	Porto	cash	12.5	39.7	27.2	2
2	Porto	card	18.0	39.7	27.2	2
3	Porto	card	9.2	39.7	27.2	2

(6 rows)

Key point:

- FILTER can be used inside windows, letting you compute conditional metrics per partition without losing rows.

Beyond the “Big 5”: list/text aggregates (string_agg, array_agg)

Why these matter:

- Many reports need “group → list of values” (e.g., coalition members, tags, cities).
- PostgreSQL gives you clean aggregates for this: string_agg and array_agg.

Key syntax:

- string_agg(expr, delimiter [ORDER BY ...])
- array_agg(expr [ORDER BY ...])

Example (build an ordered party list for each coalition)

```sql
WITH coalition_members AS (
  SELECT *
  FROM (VALUES
    ('0101', 'PSD.CDS', 1, 'PSD'),
    ('0101', 'PSD.CDS', 2, 'CDS-PP'),
    ('0201', 'PS.PAN',  1, 'PS'),
    ('0201', 'PS.PAN',  2, 'PAN')
  ) AS t(municipality_code, coalition_shortname, order_number, party_shortname)
)
SELECT
  municipality_code,
  coalition_shortname,
  string_agg(party_shortname, '.' ORDER BY order_number) AS members_shortnames,
  array_agg(party_shortname ORDER BY order_number)       AS members_array
FROM coalition_members
GROUP BY municipality_code, coalition_shortname
ORDER BY municipality_code, coalition_shortname;
```

Boolean & statistical aggregates: bool_and/bool_or, stddev_*, corr

Why these matter:

- Useful for “all/any” checks and quick descriptive stats in SQL.

Boolean aggregates:

- bool_and(condition) → TRUE if all rows satisfy it
- bool_or(condition) → TRUE if any row satisfies it

Statistical aggregates:

- stddev_samp, stddev_pop, var_samp, var_pop
- corr(x, y), covar_samp, covar_pop

Example (per district, validate and summarize mandates):

```sql
WITH municipality AS (
  SELECT *
  FROM (VALUES
    (1, 'Aveiro',  '0105', 9),
    (1, 'Aveiro',  '0109', 11),
    (1, 'Aveiro',  '0112', 5),
    (2, 'Beja',    '0205', 7),
    (2, 'Beja',    '0211', 7)
  ) AS t(district_id, district_name, municipality_code, mandates)
)
SELECT
  district_id,
  district_name,
  bool_and(mandates >= 5)       AS all_at_least_5,
  bool_or(mandates >= 10)       AS any_at_least_10,
  avg(mandates)                 AS avg_mandates,
  stddev_samp(mandates)         AS sd_mandates
FROM municipality
GROUP BY district_id, district_name
ORDER BY district_id;
```